System Architecture

Phase 1.4 — Venture Foundry OS

Derived from the frozen Product Constitution, PRD, Domain Model and Database Schema

1. Architectural Goals

This System Architecture translates the frozen Product Constitution — Company Thesis, Category Thesis, Product Philosophy, Product Principles — and the approved PRD, Domain Model and Database Schema into a technology-agnostic blueprint. Where a goal below is not stated verbatim in an approved artifact, it is derived directly from one, never invented independently of it. Nothing here reopens, extends, or reinterprets the Decision Freeze.

- AG1 — Capture reasoning as structured, linked data, not as documents. Functional Requirements FR-04 through FR-09 exist to turn a board's reasoning into named, typed, queryable records — Discussion Entries, Rationale, Assumptions, Alternatives, Risk and Obligation links — rather than a narrative document. The architecture must make this the only way reasoning enters the system; a free-text summary is not an acceptable substitute for the structured capture already frozen in the Domain Model (§5, §9–10).
- AG2 — Preserve traceability and immutability of the historical record end-to-end. NFR-01 (traceability) and NFR-02/INV-4 (versioning, not overwrite) are not features to add later; the architecture must make it structurally difficult to violate them — every write layer must attach an acting Person and timestamp, and no layer may expose an in-place edit path to a terminal Decision, Meeting, Evidence, Assumption, Risk, Obligation or Regulation record (Database Schema §8).
- AG3 — Make the Decision Graph the durable system of record, with every other layer subordinate to it. This is the direct architectural expression of the Product Philosophy's most load-bearing sentence: “Documents are evidence. Decisions are knowledge. AI is the interface; the memory graph is the product.” No layer defined in Section 3 may become a second place where a Decision Graph fact can be authored.
- AG4 — Support the frozen Future Expansion Boundary without building it. PRD §22 and Domain Model §2 both describe a later expansion (Compliance, Licensing, Contracts, Trade, Tax) that must be able to attach to the same Decision Memory Core without re-architecture. The architecture's obligation in V1 is to keep that seam open — principally by keeping Organization, Person and Regulation free of board-governance-specific assumptions (Domain Model §2 Shared Kernel) — while building nothing beyond the fourteen-step Beachhead Workflow (PRD §12).
- AG5 — Stay right-sized for a single-board, 50–500 employee organization. NFR-06 exists precisely to prevent the architecture from over-building for concurrency or scale the ICP (PRD §9) does not need. Every architectural choice in this document should be evaluated against this goal before any other: the simpler compliant design is the correct one.
- AG6 — Architect AI as a bounded retrieval interface, never as an authoritative source. FR-18 and the Product Philosophy both draw this line explicitly; PRD §20 separately names “AI reliability dependency” as an execution risk. The architecture must make the Decision Graph the only place a fact can originate, and the AI Retrieval Layer (Sections 3 and 7) the only place a fact already in the graph can be surfaced in natural language.

2. Architectural Principles

The eight principles below govern every architectural decision that follows. The first six are named directly in this phase's operating brief; two more (AP7, AP8) are added to satisfy two of this brief's own explicit constraints — that every service be independently implementable, and that this document remain technology-agnostic — and are flagged here rather than silently folded into the first six, consistent with how prior phases have flagged their own necessary inferences (Domain Model §1 P4; Database Schema §1 DP4).

- AP1 — Single Source of Truth. The Decision Graph, as realized by the canonical write model (Database Schema), is the only authoritative record of what happened and why. Every other layer either writes into this model through the Domain Layer or reads a derived view of it; none maintains an independent record of Decision Graph facts (Product Constraint: the Decision Graph "must remain the system of record"; Database Schema §1 DP7).
- AP2 — Separation of Concerns. Presentation, orchestration, business rules and storage are distinct layers with a single direction of dependency: Client → Application → Domain → Persistence. No layer reaches past its neighbor — the Client Layer never queries the Persistence Layer directly, and the Persistence Layer never encodes business rules, which is precisely why the Database Schema itself left rules like INV-2 to the application/trigger layer rather than a column constraint (Database Schema §7).
- AP3 — Immutable Audit History. Nothing that has become part of the historical record is overwritten or destroyed. This is realized as the versioning pattern (version_number / supersedes_id / is_current / superseded_at) on the seven tables the Domain Model designates as having an Active → Superseded or terminal lifecycle (Database Schema §8), and the no-hard-delete stance (Database Schema §9, INV-10).
- AP4 — Traceability by Design. Every write carries an acting Person and a timestamp (NFR-01), and every Relationship (graph edge) is itself a first-class, queryable fact (FR-15; Domain Model §7 DecisionGraphLinked event). Traceability is not a logging add-on; it is intrinsic to how each Aggregate is defined.
- AP5 — AI Augments, Never Replaces, Institutional Memory. The AI Retrieval Layer has no write path into the Decision Graph and no independent data store of record. It is a natural-language front end over the Search and Rationale Retrieval Services, both of which read the canonical model or a derived view of it. This directly implements the Product Philosophy's "AI is the interface; the memory graph is the product," and Domain Model §6's statement that the AI Retrieval Interface "must not bypass the Decision Graph as the system of record."
- AP6 — Derived Read Models over Canonical Write Models. Where performance or retrieval shape requires a different data shape than the canonical write model — a pre-joined rationale bundle for AI Retrieval (Database Schema §12, §14 item 9) or a full-text/semantic index (Database Schema §11) — that shape is built as a regenerable, derived read model, never as a second place where a fact can be recorded. If a derived model and the canonical model ever disagree, the canonical model wins by definition, and the derived model is rebuilt.
- AP7 — Bounded, Independently Implementable Services (flagged addition). Each Core Service (Section 4) owns exactly one Aggregate, or a fixed, named span of Aggregates already defined in the Domain Model, and communicates with other services only through Domain Events (Domain Model §7) or explicit calls through the Application Layer — never through direct access to another service's underlying tables. This lets Lovable, Devin, or any future engineer build or replace one service without

redesigning another, satisfying this phase's own constraint that every service be implementable independently.

- AP8 — Technology Neutrality (flagged addition). This document fixes responsibilities and boundaries, not frameworks, languages, or infrastructure. Any choice that would constitute a technology commitment rather than a structural one — for example, which specific indexing technology powers Search Service — is explicitly deferred to Section 12 or to implementation-phase documents, consistent with the Decision Freeze's instruction that this phase not reinterpret or extend approved scope.

3. High-Level Architecture

Seven layers compose the system, each named in this phase's operating brief. Dependencies flow in one direction only — downward, from Client toward Persistence — with the AI Retrieval and Document Generation Layers as read/compose-only branches off the Application Layer, and the External Integration Layer as an unbuilt seam. No framework, storage engine, or hosting model is prescribed for any layer below.

Layer	Primary Responsibility	Dependency Direction
Client Layer	Presents the Beachhead Workflow steps to Corporate Secretaries, Directors, External Counsel and Auditors; captures structured input; renders retrieved rationale. Never writes directly to any layer below the Application Layer.	Depends on Application Layer only
Application Layer	Orchestrates use cases, enforces workflow ordering (Meeting → Discussion → Decision → Rationale → Approval → Minutes → Decision Graph), and enforces authorization decisions before invoking Domain Layer operations.	Depends on Domain Layer and Authentication & Authorization Service
Domain Layer	Houses the Aggregates, Entities, Value Objects, Domain Services, Domain Events and Invariants exactly as frozen in the Domain Model (Phase 1.2). The only layer permitted to contain business rules.	Depends on Persistence Layer
Persistence Layer	Realizes the canonical write model exactly as frozen in the Database Schema (Phase 1.3): the 25 logical tables across Reference & Tenancy, Workflow Entities, Value Object Collections and the Relationship Layer.	Depended on by every layer above it; depends on nothing above it
AI Retrieval Layer	Provides natural-language access to already-captured rationale via the Search and Rationale Retrieval Services. Read-only with no path back into the Decision Graph.	Depends on Application Layer's Search and Rationale Retrieval Services only
Document Generation Layer	Compiles Minutes, drafts Resolutions, and compiles the Action Item register strictly from already-captured Decision Graph data, with no re-entry of information (PRD §21).	Depends on Domain Layer; writes generated outputs back through it
External Integration Layer	Names the seam for future, not-yet-approved integrations (PRD §22). Not built in V1 — present in this architecture only so later expansion does not require re-architecture.	Would depend on Application Layer only, never on Persistence Layer directly

Client Layer

The Client Layer is the only layer any human actor touches directly. It presents each Beachhead Workflow step (PRD §12) in sequence, matched to the persona performing it — Corporate Secretary, Board Director, External Legal Counsel, Auditor (PRD §10) — and captures structured input rather than free text wherever the Domain

Model defines a structured field. It holds no business logic: a client that skipped a step, or rendered a Decision as Approved without the Application Layer confirming it, would violate AP2 regardless of how it is built.

Application Layer

The Application Layer is the sole coordinator of multi-service use cases. It is responsible for sequencing — for example, ensuring a Decision cannot be created before its Meeting has at least one recorded attendee (INV-8) — and for authorization gating, by consulting the Authentication & Authorization Service before any Core Service call. It contains no persistence logic and no business invariant of its own; every rule it enforces is a rule already frozen in the Domain Model, invoked, not re-derived.

Domain Layer

The Domain Layer is the architecture's center of gravity: it is where the nine Aggregate Roots, two child Entities, six Value Objects, six Domain Services, the Domain Event list and the ten Business Invariants of the Domain Model (Phase 1.2) actually live and are enforced, unmodified. Every Core Service in Section 4 is, structurally, a thin operational wrapper around one or more of these Domain Layer constructs — the Domain Layer defines what is true; the Core Services define how an actor gets there.

Persistence Layer

The Persistence Layer realizes the Database Schema (Phase 1.3) exactly as frozen: the three-layer table structure (Reference & Tenancy; Workflow Entities and their owned Value Object collections; the Relationship Layer of associative and junction tables), the standard identity/audit columns, and the version-lineage pattern. Per AP1, this is the single source of truth for the entire system — every other layer's state is either a direct write into this layer or a read derived from it.

AI Retrieval Layer

The AI Retrieval Layer implements FR-18 as a natural-language front end over the Search Service and the Rationale Retrieval Service (Section 4). It is architecturally isolated from every write-capable Core Service — Meeting Management, Decision Capture, Decision Graph, and the three Document Generation services are all unreachable from this layer, per AP5. Its full role, boundaries and safeguards are detailed in Section 7.

Document Generation Layer

The Document Generation Layer implements the Minutes Compilation, Resolution Drafting and Action Register Compilation Domain Services (Domain Model §6). It is read-and-compose only with respect to the Decision Graph: it does not decide anything, it assembles already-decided facts into the Minutes, Resolution and Action Item outputs, each of which the Domain Model defines as generated once and immutable thereafter (Domain Model §11).

External Integration Layer

> **Deliberately unbuilt:** Per Product Constraints and PRD §18/§22, no external integration is in scope for V1. This layer is named here only to give the Future Expansion Boundary a defined attachment seam — mirroring the Domain Model's own treatment of Organization, Person and Regulation as a Shared Kernel (§2) — not to specify any connector, protocol, or partner. See Section 10.

4. Core Services

Ten Core Services are defined below. Nine map directly to services named in this phase's operating brief; the tenth, Action Register Generation, is added because it realizes an already-approved Domain Service (Domain Model §6 Action Register Compilation Service, FR-14) at the same architectural tier as Minutes and Resolution Generation — it is flagged here as an addition beyond the brief's example list, not as new product scope. A Notification Service is not defined; see the note at the end of this section.

Core Service	Traces To	Depends On
Authentication & Authorization	NFR-03; Domain Model §13, P6	Person and Organization Aggregates
Meeting Management	FR-01–FR-03; Domain Model §3 Meeting Aggregate, INV-8	Authentication & Authorization Service
Decision Capture	FR-04–FR-11; Domain Model §3 Decision Aggregate, §6 Decision Approval Eligibility Service, INV-1, INV-2, INV-6, INV-8	Meeting Management Service; Authentication & Authorization Service
Decision Graph	FR-15; Domain Model §6 Decision Graph Linking Service, §9–10, INV-9	Every service that produces or references an Entity
Search	FR-16 (search half); Database Schema §11	Persistence Layer's derived, text-searchable read model
Rationale Retrieval	FR-16 (retrieve half); Domain Model §6 Rationale Retrieval Service; Database Schema §12	Decision Graph Service; Persistence Layer
Minutes Generation	FR-12; Domain Model §6 Minutes Compilation Service	Meeting Management Service; Decision Capture Service
Resolution Generation	FR-13; Domain Model §6 Resolution Drafting Service, INV-3	Decision Capture Service; Decision Graph Service
Action Register Generation	FR-14; Domain Model §6 Action Register Compilation Service, INV-7	Decision Capture Service; Decision Graph Service
Audit & Traceability	NFR-01, NFR-02, NFR-05; Domain Model INV-4; Database Schema §1, §7, §8, §10	Every write-producing service

Authentication & Authorization

Field	Detail
Responsibility	Establishes acting-Person identity for every request and enforces Organization-scoped confidentiality (NFR-03, Domain Model P6, INV-5) and PersonRole-based permissions (Domain Model §8, §13 Ownership Boundaries) — for example, that Meeting creation is authored by a Corporate Secretary, per §13.
Inputs	Credential / session context. The mechanism is deliberately unspecified — this document does not assume any particular authentication technology.

Field	Detail
Outputs	An authenticated Person + Organization + Role context, passed to every other Core Service before it acts.

Meeting Management

Field	Detail
Responsibility	Owns the Meeting Aggregate: creation (FR-01), agenda upload (FR-02) and supporting-document upload (FR-03) as meeting-level Evidence, and the attendee list that INV-8 requires before a Decision may be created against the Meeting.
Inputs	Meeting date, meeting type, Organization, attending Persons, agenda document, supporting documents.
Outputs	The Meeting Aggregate (status = Open); MeetingCreated and EvidenceAttached Domain Events; calls to Decision Graph Service for each SUPPORTED_BY edge.

Decision Capture

Field	Detail
Responsibility	Owns the Decision Aggregate across its full lifecycle: creation against a Meeting and agenda item (FR-05), Discussion Entries (FR-04), Rationale (FR-06), Assumptions (FR-07), Alternatives (FR-08), Risk/Obligation links (FR-09), Votes (FR-10) and Approvals (FR-11); enforces the DecisionStatus state machine (Domain Model §11) via the Decision Approval Eligibility Service.
Inputs	Meeting and agenda-item reference, discussion entries, rationale text, assumption/risk/obligation references, vote records, approval records — each attributed to an acting Person.
Outputs	Decision Aggregate state transitions (Proposed → UnderDiscussion → Voted → Approved | Rejected); DecisionProposed, DiscussionRecorded, RationaleRecorded, AssumptionLinked, AlternativeRecorded, RiskAccepted / ObligationLinked, VoteCast, DecisionApproved / DecisionRejected Domain Events.

Decision Graph

Field	Detail
Responsibility	Creates and validates every Relationship (graph edge) among the eleven Entities, enforcing INV-9: both ends exist, are not superseded, and the pair is one of the fourteen permitted Relationship Types (Domain Model §9). This is the architectural realization of Database Schema DP7 — a single service owns edge creation regardless of which table ultimately stores it.
Inputs	Two Entity references and a named Relationship Type, submitted by whichever Core Service just completed the write on one or both ends.
Outputs	A Relationship edge, realized in the Persistence Layer as a direct foreign key, an associative table, or a junction table (Database Schema §4–§6); a DecisionGraphLinked Domain Event.

Search

> **Scope note:** FR-16 reads: “User can search and retrieve the rationale behind any past Decision” — two verbs. Search Service implements the first (locating candidate Decisions); Rationale Retrieval Service implements the second (compiling the located Decision's full rationale). This split is an architectural refinement of one approved requirement, not a new capability, and is named explicitly in this phase's own service list.

Field	Detail
Responsibility	Locates candidate Decisions (and, secondarily, Meetings, Evidence, Assumptions, Risks and Obligations) matching a keyword or structured filter, scoped to the requester's Organization (NFR-03).
Inputs	A keyword or structured query; Organization scope; optional filters (date range, status, Person).
Outputs	A ranked or matching set of Entity references, drawn from a derived, text-searchable read model (AP6) over the columns named in Database Schema §11, rather than the canonical tables directly.

Rationale Retrieval

Field	Detail
Responsibility	Given one located Decision, compiles its full connected rationale — Discussion, Assumptions, Alternatives, Risks, Obligations, Evidence, Approvals — via the fixed, bounded join list defined in Database Schema §12.
Inputs	A Decision reference, from Search Service or direct navigation.
Outputs	A rationale bundle: Discussion Entries, Alternatives, Votes, Approvals, and linked Assumption / Risk / Obligation / Evidence / Regulation records. May be served from the optional derived “rationale bundle” cache (Database Schema §12, §14 item 9) where implemented — see Section 12.

Minutes Generation

Field	Detail
Responsibility	Compiles a Meeting's Discussion, Decisions, Votes and Approvals into the Minutes Value Object (Domain Model §5, §6); transitions the Meeting Open → Concluded (Domain Model §11).
Inputs	A Meeting reference. No new information is entered — the output is compiled entirely from data already captured by Meeting Management and Decision Capture, per PRD §21's “no re-entry of information” acceptance criterion.
Outputs	The Minutes Value Object; a MinutesGenerated Domain Event.

Resolution Generation

Field	Detail
Responsibility	Generates a Resolution Entity from an Approved Decision, enforcing INV-3 (decision.status = Approved) before generation is permitted (Domain Model §6 Resolution Drafting Service).
Inputs	An Approved Decision reference.
Outputs	A Resolution Entity, generated once and immutable thereafter (Domain Model §11); a RESULTS_IN edge via Decision Graph Service; a ResolutionGenerated Domain Event.

Action Register Generation

> **Addition beyond the brief's example list:** This service is included because it realizes an already-approved Domain Service (Domain Model §6 Action Register Compilation Service, FR-14) at the same architectural tier as Minutes and Resolution Generation — the Beachhead Workflow's own step 12 names “Generate action register” alongside minutes and resolutions (PRD §12). Its absence from this phase's illustrative service list is treated as an omission of the example, not an instruction to exclude an already-approved output.

Field	Detail
Responsibility	Compiles Action Items generated by one or more Decisions within a Meeting into a register, enforcing INV-7 (each Action Item RESULTS_IN from exactly one Decision, ASSIGNED_TO exactly one Person).
Inputs	One or more Decision references requiring follow-up.
Outputs	Action Item Entities; RESULTS_IN and ASSIGNED_TO edges via Decision Graph Service; an ActionItemGenerated Domain Event.

Audit & Traceability

Field	Detail
Responsibility	A cross-cutting service, not a workflow step: ensures NFR-01 and NFR-02 are enforced at write time by attaching created_at / created_by_person_id to every write (Database Schema §1, §10) and enforcing the version-lineage rule — exactly one is_current = true row per lineage, corrections as new versions, never overwrites (INV-4) — a rule no single foreign key alone can express (Database Schema §7).
Inputs	Every write submitted by every other Core Service.
Outputs	No independent output of its own; it validates and decorates writes made elsewhere, and rejects a write that would violate NFR-01, NFR-02 or NFR-05.

> **Notification Service — omitted:** The PRD's Functional Requirements (§13) do not name a notification capability, and no Domain Event in the Domain Model (§7) is described as triggering an outbound notification. Per the Product Constraint that no workflow, object or output beyond the frozen V1 list may be added without founder approval, a Notification Service is not defined here. The Domain Event list is

> structured so that a future notification subscriber could attach without redesigning any Core Service above — but building that subscriber is out of scope for V1.

5. Request and Workflow Flows

The five flows below trace the Beachhead Workflow (PRD §12) through the layers and services defined in Sections 3–4. Each is a sequence description, not implementation code, and each ends in the specific PRD §21 Acceptance Criterion it satisfies.

Flow 1 — Meeting Creation (FR-01–FR-03)

- Step 1 — Client Layer submits meeting date, type, Organization and attending Persons to the Application Layer.
- Step 2 — Application Layer authenticates the request via Authentication & Authorization Service and confirms the acting Person's role permits creating a Meeting for that Organization (Domain Model §13).
- Step 3 — Application Layer invokes Meeting Management Service, which creates the Meeting Aggregate in the Domain Layer with status = Open.
- Step 4 — Meeting Management Service raises MeetingCreated; Audit & Traceability Service attaches created_at / created_by_person_id; the Persistence Layer commits the Meeting row.
- Step 5 — Client Layer requests agenda / supporting-document upload; Meeting Management Service records each as Evidence and calls Decision Graph Service to create the SUPPORTED_BY edge; EvidenceAttached is raised for each upload.

Result: the Meeting exists, Open, with attendees and linked Evidence — satisfying the PRD §21 Meeting & Documents acceptance criterion.

Flow 2 — Decision Capture (FR-04–FR-09)

- Step 1 — Within an Open Meeting with at least one recorded attendee (INV-8), Client Layer requests Decision creation against an agenda item.
- Step 2 — Application Layer invokes Decision Capture Service, which creates the Decision Aggregate (status = Proposed) and MADE_AT-links it to the Meeting (INV-1).
- Step 3 — As discussion proceeds, Client Layer submits Discussion Entries; Decision Capture Service appends each as a Discussion Entry Value Object, attributed to its contributing Person, and the Decision status moves to UnderDiscussion.
- Step 4 — Client Layer submits Rationale, Assumptions, Alternatives, and Risk/Obligation links; Decision Capture Service records each and calls Decision Graph Service to create the corresponding BASED_ON / ACCEPTS / SUBJECT_TO / GOVERNED_BY edges.
- Step 5 — Each write raises its corresponding Domain Event (RationaleRecorded, AssumptionLinked, AlternativeRecorded, RiskAccepted / ObligationLinked).

Result: the Decision exists with rationale, alternatives, assumptions and risk/obligation context attached, independent of its eventual outcome — satisfying the PRD §21 Discussion & Decision acceptance criterion.

Flow 3 — Decision Approval (FR-10–FR-11)

- Step 1 — Client Layer submits Votes (For / Against / Abstain) from each voting Person; Decision Capture Service records each as a Vote Value Object, enforcing INV-6, and calls Decision Graph Service for the CAST_VOTE_ON edge; the Decision status moves to Voted.

- Step 2 — Client Layer requests Approval; Decision Capture Service invokes the Decision Approval Eligibility Service, which checks INV-2 — Rationale present, at least one Vote, at least one Approval — before allowing the status transition.
- Step 3 — On success, Decision Capture Service records the Approval Value Object, transitions the Decision to Approved (or Rejected, per Domain Model §11's modeling note), and calls Decision Graph Service for the APPROVED_BY edge; DecisionApproved or DecisionRejected is raised.
- Step 4 — Per INV-4, all Decision content becomes immutable at this point; any later correction is architected as a new version, never an in-place edit (Database Schema §8).

Result: the Decision reaches a terminal, immutable state — satisfying the PRD §21 Vote & Approval acceptance criterion.

Flow 4 — Resolution Generation (FR-12–FR-14)

- Step 1 — Once one or more Decisions in a Meeting are Approved, Client Layer requests output generation.
- Step 2 — Application Layer invokes Minutes Generation Service, which compiles the Minutes Value Object from already-captured data and transitions the Meeting Open → Concluded; MinutesGenerated is raised.
- Step 3 — For each Approved Decision, Application Layer invokes Resolution Generation Service, which enforces INV-3 before generating the Resolution Entity and calling Decision Graph Service for the RESULTS_IN edge; ResolutionGenerated is raised.
- Step 4 — For each Decision requiring follow-up, Application Layer invokes Action Register Generation Service, which generates Action Item Entities under INV-7 and the corresponding RESULTS_IN / ASSIGNED_TO edges; ActionItemGenerated is raised.

Result: Minutes, Resolution(s) and an Action Item register exist, generated entirely from already-captured data with no re-entry of information — satisfying the PRD §21 Output Generation acceptance criterion.

Flow 5 — AI Rationale Retrieval (FR-16, FR-18)

- Step 1 — An authorized user submits a natural-language query to the AI Retrieval Layer.
- Step 2 — AI Retrieval Layer authenticates the request and constrains the query to the requester's Organization scope (NFR-03, INV-5) before any retrieval occurs — this scoping happens outside the AI model, not inside a prompt (Section 7).
- Step 3 — AI Retrieval Layer calls Search Service to translate the query into a candidate set of matching Decisions, operating over the derived, text-searchable read model rather than the canonical tables directly.
- Step 4 — For the selected Decision(s), AI Retrieval Layer calls Rationale Retrieval Service to compile the full connected rationale bundle (Database Schema §12 bounded fan-out).
- Step 5 — AI Retrieval Layer constructs a response strictly from the retrieved bundle, attributing every claim to its source record, and returns it to the Client Layer, performing no write of its own back into the Decision Graph.

Result: the user receives rationale traceable back to specific Decision Graph records — satisfying the PRD §21 Decision Graph & Retrieval acceptance criterion and Domain Model §6's requirement that the AI interface not bypass the graph as system of record.

6. Security Architecture

Security is modeled here at the level of responsibility and boundary, not mechanism — no specific authentication technology, storage engine, or hosting capability is assumed, per this phase's constraints.

Authentication

Authentication establishes acting-Person identity for every request; the mechanism by which it is issued is deliberately unspecified. Every layer below the Client Layer trusts only the Person + Organization + Role context established at this step — the Client Layer never asserts its own identity claims to the Domain Layer (AP2).

Authorization

Authorization is role-based, per the PersonRole enumeration (Domain Model §8) and the Ownership Boundaries table (Domain Model §13): Corporate Secretary authors Meetings, Persons and Evidence; Directors and External Counsel contribute to Decisions; Resolution, Action Item and Minutes are system-generated, authored by no individual. Architecturally, this means Minutes Generation, Resolution Generation and Action Register Generation must run under a system principal distinct from the delegated permissions of the user who requested generation — the request is authorized by role, but the generated content's authorship is the system of record itself, per §13.

Tenant Isolation

Organization is the tenancy boundary (Domain Model P6, NFR-03). Evidence, Assumption, Risk and Obligation carry organization_id directly (Database Schema DP5); Person and Meeting reach Organization in exactly one hop. Every Core Service query must be implicitly scoped by the acting Person's Organization before any other filter is applied — this is an architectural requirement on every service in Section 4, not an application-level afterthought. Regulation is the sole exception, architected as shared reference data reachable from every Organization (Domain Model §13).

Audit Logging

Audit logging is not a bolt-on log stream; it is the version lineage and created_by / created_at columns themselves (AP3, AP4). Database Schema §10 states this directly: the version lineage is the audit log for versioned tables, and the single created_at / acting-Person pair is the complete traceability record for every other table.

Confidentiality Boundaries

INV-5 — Evidence, Assumption, Risk and Obligation are reusable only within one Organization's Decision Graph — is enforced at Decision Graph Service, at edge-creation time, as a hard boundary. An edge request that would link two Entities from different Organizations must be rejected by Decision Graph Service itself, not merely hidden by a downstream query filter, since INV-9 already requires every edge to connect only permitted, existing Entities.

7. AI Architecture

FR-18 and the Product Philosophy (“AI is the interface; the memory graph is the product”) together define AI's role precisely: an interpretive layer over data that already exists, never a source of new facts. The six subsections below specify how that boundary is held architecturally, not just declared.

Inputs

A natural-language query, plus the authenticated Person / Organization / Role context established by Authentication & Authorization Service. The AI Retrieval Layer never receives raw, unscoped access to the full Decision Graph as an input.

Retrieval Boundaries

The AI Retrieval Layer may read only through Search Service and Rationale Retrieval Service, both of which are themselves Organization-scoped (Section 6). It has no direct connection to the Persistence Layer and no path to any write-capable Core Service — Meeting Management, Decision Capture, Decision Graph, and the three Document Generation services are all unreachable from this layer, per AP5.

Prompt Construction Principles

The query given to the underlying language model is assembled from exactly two things: the person's natural-language question, and the already-retrieved, Organization-scoped rationale bundle. The Organization boundary is enforced before the AI model ever sees the data — by Search and Rationale Retrieval Service scoping, not by asking the model to decide what it may or may not reveal.

Hallucination Mitigation

The AI Retrieval Layer is prohibited from answering from the model's general training knowledge. Every factual claim in a response must be grounded in a specific record returned by Rationale Retrieval Service. Where the retrieved bundle does not contain an answer, the correct behavior is to state that the Decision Graph has no recorded rationale on that point — never to infer or fabricate one.

Source Attribution

Every response must be traceable to the specific Decision, Discussion Entry, Assumption, Risk, Obligation or Evidence record it drew from, reusing the same Relationship edges (Domain Model §9–10) that already make the graph traceable. Attribution is not a separate feature bolted onto the response; it is the graph's own structure surfaced to the reader, which is what FR-16's “retrieve” requirement already demands independent of the AI interface.

Why AI Never Becomes the System of Record

The AI Retrieval Layer has no Aggregate of its own in the Domain Model, no table of its own in the Database Schema, and no Domain Event that only it can raise. Every fact it surfaces already exists, attributed and versioned, in the canonical write model before the AI Retrieval Layer ever reads it (AP1, AP5). This is an architectural guarantee, not a policy statement: there is structurally no path by which the AI Retrieval Layer could originate a fact the rest of the system does not already have.

8. Scalability & Performance Considerations

NFR-06 explicitly right-sizes this architecture for single-board, 50–500 employee organizations, not large-enterprise concurrency — every consideration below is read against that constraint, not against generic web-scale assumptions.

- Write-path load is bursty and low-frequency: one Meeting, cycling through a bounded set of Decisions, on a cadence of board meetings per Organization (typically monthly or quarterly). This does not justify write-path scaling investment beyond correctness and traceability (AG5).
- Read-path load is comparatively higher-frequency and grows over time, as historical Decisions accumulate and are queried repeatedly — directly reflected in the PRD §8 Time-to-context success metric. This asymmetry is why AP6 (derived read models) is the primary scaling lever, not write-path optimization.
- Decision INFORMED_BY Decision (Domain Model §10; Database Schema §12) is the one genuinely multi-hop relationship in the model; its recursive-traversal cost grows with the depth of reuse a mature Organization accumulates. As Decision Graph reuse (PRD §8 Success Metric) grows, this is the traversal most likely to warrant its own derived representation rather than a naive recursive query on every read.
- Because Organization is the hard isolation boundary (Section 6), per-Organization data volume — not aggregate platform volume — is the correct unit to reason about for V1; a single Organization's board cadence bounds its own Decision Graph size predictably.

9. Error Handling & Resilience

Because nothing in this system is hard-deleted (INV-10, Database Schema §9), most failure recovery is architected as forward-only correction, never as retrying a destructive operation.

- Validation failures — for example, attempting to Approve a Decision without Rationale, a Vote, or an Approval (INV-2) — are rejected inside Decision Capture Service, in the Domain Layer, before any Persistence Layer write. The Application Layer must not attempt to silently work around a failed invariant.
- A failed or mistaken write to a versioned table (Database Schema §8) is corrected by superseding it with a new version, never by retrying a destructive in-place edit.
- Retries must be idempotent with respect to supersedes_id chains: the Domain Layer, not client-side retry logic, is responsible for checking whether a correction has already been recorded before creating a new version.
- Decision Graph Service creates an edge only after both endpoint writes are confirmed — INV-9's requirement that an edge connect two existing Entities is an ordering constraint on write sequencing, not only a validation rule, so a partial-workflow failure (for example, a Meeting created but an Evidence upload that fails) cannot leave an inconsistent edge behind.
- Because Resolution and Minutes are each defined as single-generation, immutable outputs (Domain Model §11), the Document Generation Layer must check for an existing generated output before creating a new one, so that a retried generation request cannot duplicate it.
- Recovery must never compromise NFR-05 (“no silent data loss for Decisions, Approvals, or Evidence”): any failure mode that cannot be resolved without data loss must surface as an explicit error to the Application Layer, never as a silently dropped write.

10. Integration Boundaries

Per Product Constraints and PRD §18/§22, no external integration is in scope for V1. The considerations below define the discipline any future integration must respect — none of them names or designs a specific integration.

- The External Integration Layer (Section 3) exists in this architecture only to name the seam, not to specify a connector, protocol, or partner.
- The Shared Kernel named in Domain Model §2 — Organization, Person and Regulation — is the intended attachment point for future sibling Bounded Contexts (Compliance, Licensing, Contracts, Trade, Tax, per PRD §22's frozen expansion path). The architecture's obligation in V1 is only to keep these three Aggregates free of board-governance-specific assumptions that would make later reuse costly — not to build anything for them to attach to yet.
- Regulation is modeled as shared, centrally curated reference data rather than Organization-owned (Domain Model §13) — this is the one place in V1 where a future integration (for example, an external regulatory-content feed) could write into Regulation without needing to touch any Organization-scoped data, precisely because Regulation was already isolated from the confidentiality boundary for this reason.
- Any future integration must enter through the Application Layer as a new caller of existing Core Services (Section 4) — never by granting external systems direct Persistence Layer access, which would violate AP1 and AP2 as a matter of architectural principle, independent of which integration is eventually approved.

11. Traceability Matrix

Every architectural component defined in Sections 3 and 4 is mapped below to the specific point in each frozen upstream artifact that justifies its existence. Cells marked — indicate the component is a structural / orchestration concern with no direct object-level mapping in that artifact, stated explicitly rather than left blank without explanation.

Architectural Component	Product Constitution	PRD	Domain Model	Database Schema
Client Layer	Product Principle: capture reasoning at decision time	§10 Personas; §15 User Stories	— (presentation, not modeled)	—
Application Layer	Product Principle: every feature strengthens EDM	§12 Beachhead Workflow	§2 Bounded Context	—
Domain Layer	Philosophy: “Decisions are knowledge”	FR-01–FR-15	§3–§12 (Aggregates through Invariants)	§1 DP1–DP7
Persistence Layer	Philosophy: “the memory graph is the product”	FR-17	P5 Identity; §14	§1–§14 (entire document)
AI Retrieval Layer	Philosophy: “AI is the interface”	FR-18	§6 AI Retrieval Interface note	§11–§12
Document Generation Layer	Philosophy: “Documents are evidence” (inverted: outputs of decisions, not decisions themselves)	FR-12–FR-14	§6 three compilation/drafting services; §11	minutes, resolution, action_item tables (§3)
External Integration Layer	Product Vision: expansion path	§22 Future Expansion Boundary	§2 Shared Kernel	— (deliberately unbuilt)
Authentication & Authorization	— (execution concern)	NFR-03	§13 Ownership Boundaries; P6	DP5; §7
Meeting Management	—	FR-01–FR-03	§3 Meeting Aggregate; INV-8	meeting, evidence tables
Decision Capture	Philosophy: “Decisions are knowledge”	FR-04–FR-11	§3 Decision Aggregate; §6; INV-1,2,6,8	decision, decision_discussion_entry, decision_alternative, decision_vote, decision_approval
Decision Graph	Philosophy: “the	FR-15	§6 Decision Graph	§4–§6

Architectural Component	Product Constitution	PRD	Domain Model	Database Schema
	memory graph is the product"		Linking Service; §9–10; INV-9	
Search	North Star: reusable, searchable rationale	FR-16 (search half)	— (architectural refinement)	§11
Rationale Retrieval	North Star: reusable, searchable rationale	FR-16 (retrieve half)	§6 Rationale Retrieval Service	§12
Minutes Generation	—	FR-12	§6 Minutes Compilation Service	minutes table
Resolution Generation	—	FR-13	§6 Resolution Drafting Service; INV-3	resolution table
Action Register Generation	—	FR-14	§6 Action Register Compilation Service; INV-7	action_item table
Audit & Traceability	Product Principle: preserve traceability	NFR-01, NFR-02, NFR-05	INV-4	§1, §7, §8, §10

12. Architectural Risks & Deferred Decisions

The items below are unresolved by design — each is surfaced explicitly rather than silently decided, consistent with the flagging discipline established in the PRD, Domain Model and Database Schema.

#	Deferred Item	Status / What Is Needed
1	Search / AI Retrieval indexing technology (full-text vs. semantic/vector).	Database Schema §11 explicitly deferred this choice to System Architecture. This document resolves only the logical requirement — Search Service must support text-searchable and connected-graph retrieval — not the concrete engine, per the constraint against prescribing frameworks.
2	The optional “rationale bundle” derived cache (Database Schema §12, §14 item 9).	Architected here as the correct application of AP6 if and when performance requires it. Whether it is built in V1 or deferred to a later increment is not decided by this document; confirm at implementation planning.
3	Search Service as distinct from Rationale Retrieval Service.	Flagged in Section 4 as a refinement of FR-16's two verbs (search, retrieve), not new scope. Recommend confirming at governance review since it introduces a named service the Domain Model's §6 service list did not separately enumerate.
4	MeetingType enumeration values (Domain Model §8 note).	Still unconfirmed by a legal/governance advisor; carried forward unresolved from the Domain Model without change.
5	Data residency / jurisdiction (NFR-04) and SLA / uptime targets (PRD §8, §14).	Both remain open Potential Strategic Changes per the PRD's own Governance Note. This architecture makes no jurisdiction-specific or uptime-specific commitment.
6	Authentication mechanism.	Deliberately unspecified per this phase's constraints (no APIs, no assumed platform-specific capabilities). Section 6 defines only the role/session context every layer must trust, not how it is issued.
7	Evidence file storage mechanism.	The file_reference / content_reference / compiled_content columns (Database Schema §3) are pointers only; this document does not specify a file-hosting mechanism, consistent with the prohibition on defining deployment infrastructure.
8	Notification capability.	Confirmed omitted (Section 4). Domain Events (Domain Model §7) are structured to support a future subscriber without redesign, but building one requires founder approval per the Product Constraints.
9	External Integration Layer connectors.	No specific future integration (e.g., regulator e-filing, e-signature for Resolutions) is named or designed. Section 10 defines only the boundary discipline such an integration must respect.
10	Regulation curation process (manual vs. automated feed).	Unresolved in both the Database Schema (§14 item 8) and this architecture; also affects whether the External Integration

#	Deferred Item	Status / What Is Needed
		Layer needs a write path into Regulation sooner than into any other Aggregate.

Governance Note

This completes the Phase 1.4 deliverable. This document derives entirely from the approved PRD, Domain Model and Database Schema; none was revisited or modified, per the operating instructions for this phase. Two architectural principles (AP7, AP8), one service (Action Register Generation), and one service split (Search versus Rationale Retrieval) were made explicit as flagged additions beyond this phase's example lists, each traced to an already-approved requirement rather than to new scope. Ten items were surfaced as open architectural risks or deferred decisions (Section 12) rather than silently resolved, the most consequential being the still-open choice of Search/AI Retrieval indexing technology that the Database Schema itself deferred to this document. Per Venture Foundry OS operating instructions, this document now stops and awaits governance approval before proceeding to API Contracts.